
SUNTA: Hierarchical Video Prediction with Surprise-based Chunking

Tomoshi Iiyama Masahiro Suzuki Yutaka Matsuo
The University of Tokyo
{iiyama, masa, matsuo}@weblab.t.u-tokyo.ac.jp

Abstract

Hierarchical state-space models (HSSMs) offer a promising approach to long-horizon prediction by segmenting sequences into temporal chunks. However, their performance hinges on how chunk boundaries are determined. While prior HSSMs typically rely on fixed-length chunking or similarity-based boundary detection, these methods often misalign with the intrinsic temporal structure of the data. We argue that chunking should instead be driven by prediction errors, which more directly indicate when longer-range context becomes necessary. Nevertheless, integrating surprise-based chunking into HSSMs introduces critical challenges, including hierarchical collapse during end-to-end training and the absence of surprise signals during open-loop prediction. To address these issues, we propose Surprise-based Nested Temporal Abstraction (SUNTA), a method that employs a decoupled training strategy to preserve surprise signals and uses internal inconsistency as a top-down surprise metric to determine chunk boundaries within imagined rollouts. Experiments on video prediction tasks in 2D and 3D environments demonstrate that SUNTA outperforms baselines, uniquely maintaining accurate predictions over 250 timesteps, whereas all baselines degrade within the first 10 timesteps.

1 Introduction

Predicting future environmental states is a cornerstone of intelligence [21, 23], motivating research on world models that endow agents with internal predictive models [9, 12, 28] and substantially enhance planning and decision-making [14, 15, 17, 18, 19, 20]. A central challenge for such models is to support long-horizon prediction: since complex tasks often require foresight to anticipate distant consequences [16, 37, 42], capturing long-term dynamics is essential for practical world models.

A promising route toward long-horizon prediction is hierarchical state-space models (HSSMs) [11, 25, 26, 39, 41, 50], which decompose sequences into temporal *chunks* and learn dynamics across multiple timescales. Crucially, performance depends on chunk boundaries: they should align with intrinsic temporal structure so that high-level dynamics are easier to model; misalignment forces the higher level to explain entangled transitions and obscures regularities even when the underlying dynamics are simple¹.

Nevertheless, existing HSSMs typically fix chunk lengths [39, 41] or constrain lengths toward a preset value [26]. Recent work, such as VPR [50], instead infers boundaries from observational changes such as visual shifts (i.e., similarity-based chunking). However, such cues can fail in practice: important contextual shifts in the underlying dynamics may occur without visual salience, while superficial visual changes may appear without semantic transitions.

¹For example, predicting a sentence becomes unnecessarily difficult when it is segmented into arbitrary substrings (Iforg → otmyu → mbrel → laont → hetra → in) rather than into syntactically meaningful units (I → forgot → my → umbrella → on → the → train)

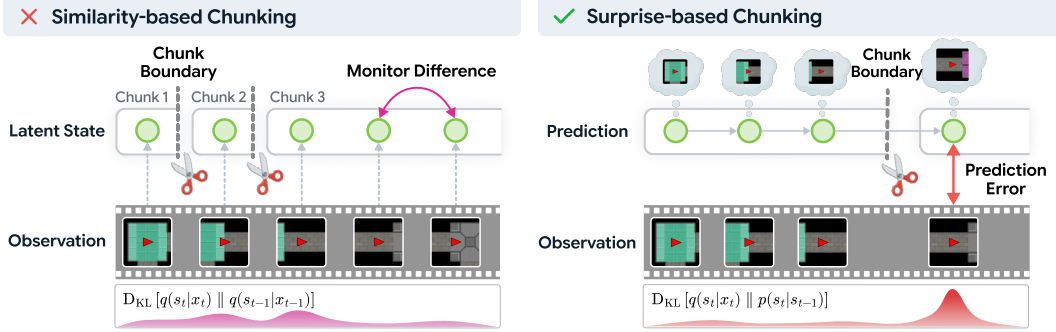


Figure 1: **Similarity-based chunking vs. surprise-based chunking.** Similarity-based chunking monitors observational changes and may over-segment superficial appearance changes or miss non-salient semantic transitions. Surprise-based chunking cuts at peaks of prediction error, highlighting shifts in the latent dynamics.

We argue that temporal abstraction should be driven by prediction errors, or *surprise*, within the internal world model rather than visual changes, since surprise reflects shifts in the underlying dynamics rather than superficial appearance differences (see Fig. 1): chunks should begin and end where prediction error spikes, marking moments where higher-level abstraction should intervene.

The concept of surprise-based chunking dates back to early sequence modeling [40] and has been revisited for video understanding [1, 34]. However, existing methods are mainly limited to *recognition* settings. While they segment data into chunks, they do not exploit those chunks for *generation*, because naively introducing a surprise-based boundary criterion into HSSMs can lead to failure. We identify two critical gaps that must be addressed to integrate surprise-based chunking into HSSMs:

Gap 1: Hierarchical collapse under surprise minimization. In a naive implementation, low-level prediction errors are used to mark chunk boundaries, triggering transitions of high-level states. However, the low level becomes *less* surprised as the high-level model improves, because effective top-down conditioning makes the low-level dynamics easier to predict. This erases the very surprise signals that defined the chunks, causing the hierarchy to cycle between emergence and collapse.

Gap 2: Missing surprise during open-loop generation. Surprise signals rely on comparing predictions to external observations, limiting their use to the *inference* or recognition process. In open-loop rollouts of world models, we would like to leverage chunking to enable longer-horizon prediction; however, observations are inherently unavailable, so we cannot compute surprise-based chunk switches.

To address Gap 1, we treat each level as an independent module and optimize them *sequentially*, so that improvements at the high level do not suppress the low-level surprise signal used for chunk discovery. For Gap 2, we enable open-loop chunk detection by replacing bottom-up, observation-based surprise with a *top-down* surprise signal derived from the high-level representation during generation. We define this top-down surprise as the mismatch between the evolving low-level rollout and the current high-level context; as the mismatch grows, it triggers chunk termination and enables dynamic chunk detection.

Guided by these insights, we present an HSSM called **Surprise-based Nested Temporal Abstraction** (SUNTA). To prevent hierarchical collapse, we adopt inter-level prediction: just as the low-level model predicts observations from the environment, the high-level model predicts low-level latents chunk by chunk, treating them as its observations. By training each level sequentially, SUNTA achieves stable optimization while maintaining surprise-driven hierarchical structures. During open-loop generation, SUNTA uses this top-down surprise to determine chunk boundaries without observations.

Finally, at test time (online inference or generation), the model must infer a high-level chunk representation from a *partial* low-level chunk sequence, whereas during training it observes the *entire* chunk. To bridge this train–test mismatch, we introduce *temporal pattern completion* regularization, training the high-level encoder to recover the chunk representation from incomplete low-level inputs and improving robustness to partial context.

We evaluate SUNTA on three video datasets: a simple synthetic dataset with a moving ball, a 2D environment where an agent navigates across multiple rooms, and a 3D maze environment. We

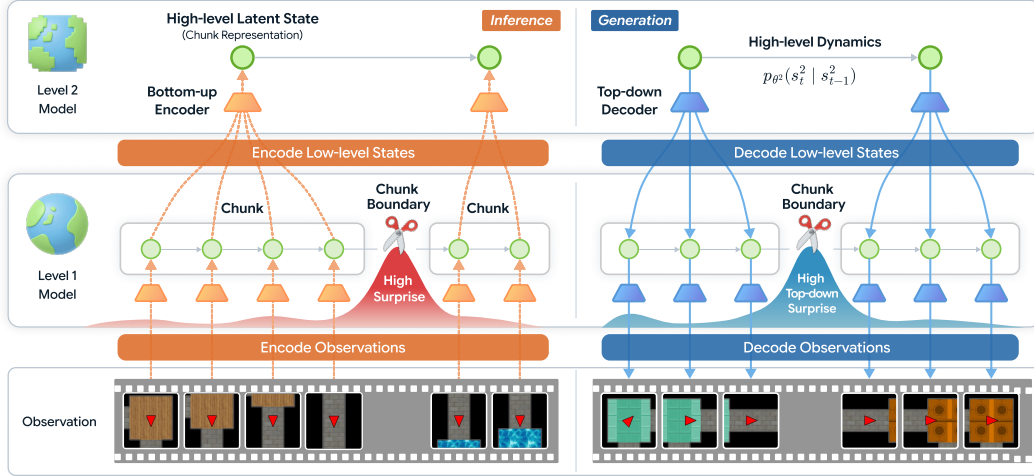


Figure 2: **Overview of SUNTA.** During inference (left), observations are encoded into low-level states and segmented into chunks at points of high prediction error. A bottom-up encoder aggregates each chunk into a single latent state for high-level dynamics modeling. During generation (right), the high-level model produces high-level states that are decoded into low-level sequences. Chunk boundaries are detected via top-down surprise, which measures the mismatch between low-level rollouts and high-level context, enabling dynamic chunking without ground-truth observations.

evaluate the surprise-based criterion for accurately discovering meaningful temporal boundaries in videos and demonstrate the effectiveness of SUNTA against 5 baseline models; in particular, SUNTA is the only model that maintains accurate predictions for 250 timesteps, whereas all baselines degrade within the first 10 timesteps.

2 Preliminaries

State-space models SSMS assume that observed sequences $x_{1:T}$ are generated by an underlying, hidden process of latent states $s_{1:T}$. The model is defined by transition dynamics $p(s_t | s_{t-1})$, describing how the state evolves at each timestep t , and an observation model $p(x_t | s_t)$ that generates the observation conditioned on the latent state.

Hierarchical state-space models HSSMs extend standard SSMS by introducing multi-level latent states that operate at multiple time scales. They decompose a sequence of observations $x_{1:T}$ into temporal segments, referred to as chunks. The low-level latent state $s_t^{(1)}$ operates at the frequency of raw observations (e.g., every video frame), while the high-level latent state $s_t^{(2)}$ remains constant within a chunk and is updated only at chunk boundaries. To formalize this temporal structure, we represent chunk boundaries using a binary variable $m_t \in \{0, 1\}$, indicating whether a new chunk starts at time step t [26]. If a boundary is not activated ($m_t = 0$), the high-level state is copied from the previous time step, persisting within the same chunk. If a boundary is activated ($m_t = 1$), the high-level state transitions and a new chunk begins.

The critical difference among existing HSSMs lies in how the chunk boundary m_t is determined. Existing approaches can be categorized into three main strategies:

- **Fixed interval:** Fixes a chunk length H as a hyperparameter, deterministically setting $m_t = 1$ if and only if $t \bmod H = 0$ [39, 41].
- **Learning under constraints:** Treats m_t as a latent variable inferred from observations via a variational posterior $q(m_{1:T} | x_{1:T})$ [25, 26].
- **Similarity-based:** Determines boundaries non-parametrically by setting $m_t = 1$ when the difference between successive latent representations exceeds a threshold [50].

In this paper, we instead explore a surprise-based approach, where chunk boundaries are determined by the magnitude of prediction error in the low-level latents.

3 Surprise-based Nested Temporal Abstraction

We introduce surprise-based nested temporal abstraction (SUNTA), a two-level HSSM that dynamically constructs its temporal hierarchy based on surprise signals.

Low-level model (level 1) We build our model upon the recurrent state space model (RSSM) [13], a backbone widely used in many HSSMs. RSSM learns compact latent-state representations from high-dimensional observations using variational autoencoders, while modeling latent dynamics with recurrent neural networks (RNNs). We consider an offline dataset consisting of sequences of observations $(x_1, \dots, x_T)$. We denote the low-level latent state at timestep t as $s_t^{(1)}$, which is a stochastic variable parameterized as a diagonal Gaussian. The low-level model of SUNTA, parameterized by θ_1 , consists of the following components:

$$\text{Posterior:} \quad s_t^{(1)} \sim q_{\theta_1}(s_t^{(1)} \mid s_{<t}^{(1)}, x_t), \quad (1)$$

$$\text{Prior:} \quad \hat{s}_t^{(1)} \sim p_{\theta_1}(s_t^{(1)} \mid s_{<t}^{(1)}), \quad (2)$$

$$\text{Decoder:} \quad \hat{x}_t \sim p_{\theta_1}(x_t \mid s_t^{(1)}). \quad (3)$$

Here, the posterior is implemented with an autoregressive gated recurrent unit (GRU) that summarizes the latent history $s_{<t}^{(1)}$, and is conditioned on an observation embedding obtained by a convolutional neural network (CNN) encoder applied to x_t . The prior uses a GRU to predict the next latent state from the same history, without access to x_t . The decoder reconstructs the observation from the stochastic latent state $s_t^{(1)}$.

High-level model (level 2) Unlike typical HSSMs that treat all levels as a single neural network, SUNTA employs a decoupled design where each level operates independently. Analogous to the standard RSSM that models observations from the environment, the high-level model in SUNTA treats the already learned low-level latents as *observations*. These low-level latents are segmented into chunks determined by the boundaries m_t , which are identified using the surprise criterion regarding the low-level predictions. For each timestep t , we denote by $\tau(t)$ the starting timestep of the chunk to which t belongs, so that the interval $[\tau(t), t]$ corresponds to the current chunk:

$$\tau(t) = \max\{t' \mid t' \leq t \wedge m_{t'} = 1\}. \quad (4)$$

Within a chunk, the high-level model learns a latent state $s_t^{(2)}$ that compresses the low-level sequence $s_{\tau(t):t}^{(1)}$ into a single representation. The high-level model, parameterized by θ_2 , comprises the following components:

$$\text{Posterior:} \quad s_t^{(2)} \sim q_{\theta_2}(s_t^{(2)} \mid s_{<t}^{(2)}, s_{\tau(t):t}^{(1)}), \quad (5)$$

$$\text{Prior:} \quad \hat{s}_t^{(2)} \sim p_{\theta_2}(s_t^{(2)} \mid s_{<t}^{(2)}), \quad (6)$$

$$\text{Top-down Decoder:} \quad \tilde{s}_{\tau(t):t}^{(1)} \sim p_{\theta_2}(s_{\tau(t):t}^{(1)} \mid s_t^{(2)}). \quad (7)$$

Here, q_{θ_2} is implemented with a GRU-based encoder that aggregates the low-level latent sequence within a chunk and outputs the parameters of the posterior over $s_t^{(2)}$. The prior p_{θ_2} is implemented as a GRU that predicts the next high-level state from past high-level states without access to low-level latents. The top-down decoder reconstructs the corresponding low-level latent sequence from $s_t^{(2)}$.

Surprise-based chunking We use the prediction error to determine chunk boundaries: the high-level model is activated only when the low-level model fails to predict the next input. In probabilistic generative models, the prediction error can be quantified as Bayesian Surprise [33], measured by the Kullback-Leibler (KL) divergence between the posterior and prior distributions. We monitor the surprise measure in the low-level model at every time step, marking chunk boundaries whenever sharp increases in surprise are detected. During *inference*² (recognition phase), we employ the following surprise criterion to identify chunk boundaries m_t :

$$D_{\text{KL}} \left[q_{\theta_1}(s_t^{(1)} \mid s_{<t}^{(1)}, x_t) \parallel p_{\theta_1}(s_t^{(1)} \mid s_{<t}^{(1)}) \right] > \tau_{\text{inf}}. \quad (8)$$

²Throughout this paper, we use the term “inference” in the classical generative-modeling sense of recognizing latent variables given observations (e.g., variational inference).



Figure 3: **Hierarchical structures detected on the Bouncing Ball dataset.** Each colored block denotes a chunk identified by the corresponding model. Ground-truth boundaries occur when the ball bounces and changes color. SUNTA precisely identifies these transitions, while VPR and VTA produce fragmented over-segmentation, and LOVE fails to detect meaningful boundaries altogether.

The threshold τ_{inf} determines the sensitivity for detecting spikes. While adaptive thresholds like moving averages could be used, in practice, we found that peak-detection algorithms that find local maxima in time series (e.g., SciPy’s `find_peaks`) work more robustly than simple moving-average thresholds. See Section D for implementation details and sensitivity analysis.

Level-wise training A primary obstacle in learning HSSMs lies in the circular dependencies between hierarchical levels that arise when attempting end-to-end learning involving both top-down and bottom-up processes within a Bayesian framework [31]. Consequently, many existing HSSMs bypass explicit bottom-up inter-level inference, instead inferring latent states at all levels conditioned solely on the raw environmental observations [25, 26, 39, 50]. SUNTA adopts an alternative strategy: we train the hierarchy *sequentially*, optimizing each level independently before moving to the next. This decoupled optimization avoids circular dependencies and ensures that low-level latents and their boundaries are learned without being influenced by higher-level learning.

Open-loop chunk detection via top-down surprise During rollout, the high-level state provides chunk-level context while the top-down decoder generates the concrete low-level dynamics conditioned on it. However, surprise signals normally require external observations, and are therefore unavailable in open-loop rollouts; as a result, we cannot compute bottom-up, observation-based surprise to decide when to switch chunks. We address this by introducing a *top-down surprise* signal that measures the mismatch between the evolving low-level rollout and the current high-level context. Intuitively, as the rollout drifts beyond what the current high-level state represents, top-down surprise grows and signals that the chunk should terminate. Specifically, at each step we first decode the current high-level state using the top-down decoder to generate the low-level latent state $\tilde{s}_t^{(1)}$. We then employ the following top-down surprise criterion to identify chunk boundaries:

$$D_{\text{KL}} \left[q_{\theta_2} \left(s_t^{(2)} \mid s_{<t}^{(2)}, \tilde{s}_{\tau(t):t}^{(1)} \right) \parallel p_{\theta_2} \left(s_t^{(2)} \mid s_{<t}^{(2)} \right) \right] > \tau_{\text{gen}}, \quad (9)$$

where τ_{gen} controls the sensitivity for detecting chunk switches.

Temporal pattern completion (TPC) A key challenge in HSSMs is a train–test mismatch at higher levels. During the training of a high-level model, the bottom-up encoder has access to the *entire* lower-level chunk to infer the corresponding high-level latent state. However, during online inference or generation, the model must perform inference based on only a *partial* chunk observed up to the current timestep. This mismatch can yield inaccurate high-level representations from incomplete patterns, degrading the model’s ability to reconstruct inputs or predict future transitions accurately. To mitigate this, we introduce a regularizer that encourages the representation inferred from a partial chunk to match the more informative representation inferred from the complete sequence. Concretely, we compare two posteriors over the high-level state: a *complete* posterior inferred from the entire lower-level chunk and a *partial* posterior inferred from a prefix of the same chunk. We minimize the discrepancy between the *complete* and *partial* posteriors with a KL penalty, and add a contrastive separation term to prevent representational collapse by enforcing a margin from a *negative* posterior. Here, $q_{\theta_2}^c(s_t^{(2)} \mid \cdot)$, $q_{\theta_2}^p(s_t^{(2)} \mid \cdot)$, and $q_{\theta_2}^n(s_t^{(2)} \mid \cdot)$ denote the posteriors inferred from the *complete* chunk, its *partial* prefix, and an unrelated (*negative*) chunk context (e.g., a different chunk/sequence),

respectively. The resulting temporal pattern completion loss is:

$$\mathcal{L}_{\text{TPC}}(\theta_2) = \text{D}_{\text{KL}}[q_{\theta_2}^c(s_t^{(2)} | \cdot) \parallel q_{\theta_2}^p(s_t^{(2)} | \cdot)] + \left[\gamma - \text{D}_{\text{KL}}[q_{\theta_2}^c(s_t^{(2)} | \cdot) \parallel q_{\theta_2}^n(s_t^{(2)} | \cdot)] \right]_+, \quad (10)$$

where $[z]_+ = \max(0, z)$ and $\gamma > 0$ is a margin hyperparameter. This encourages high-level states that are robust to missing future context while remaining discriminative across different chunks. We illustrate the train–test mismatch and our TPC procedure in Section H.

Model training As previously outlined, SUNTA employs a level-wise training strategy where each level is trained independently, proceeding from the lowest level upward. Initially, the low-level model ($l = 1$) is trained on the raw observation data. The training objective for the low-level model is equivalent to that of a standard RSSM, aiming to maximize the Evidence Lower Bound (ELBO):

$$\mathcal{L}(\theta_1) = \mathbb{E}_{q_{\theta_1}(s_{\leq T}^{(1)} | x_{\leq T})} \left[\sum_{t=1}^T \log p_{\theta_1}(x_t | s_t^{(1)}) - \sum_{t=1}^T \text{D}_{\text{KL}}(q_{\theta_1}(s_t^{(1)} | s_{<t}^{(1)}, x_t) \parallel p_{\theta_1}(s_t^{(1)} | s_{<t}^{(1)})) \right], \quad (11)$$

which maximizes reconstruction likelihood while regularizing the posterior toward the prior. After training, we freeze θ_1 and infer low-level latent sequences $s_{1:T}^{(1)}$ (and boundaries $m_{1:T}$). We then train the high-level model by treating these inferred low-level latents as *observations*. For the high-level model, the objective consists of (i) a reconstruction term for low-level latents, (ii) a KL regularizer for the high-level dynamics, and (iii) the temporal pattern completion regularizer:

$$\begin{aligned} \mathcal{L}(\theta_2) = \mathbb{E}_{q_{\theta_2}(s_{\leq T}^{(2)} | s_{\leq T}^{(1)}, m_{\leq T})} & \left[\sum_{t=1}^T \log p_{\theta_2}(s_{\tau(t):t}^{(1)} | s_t^{(2)}) \right. \\ & \left. - \sum_{t=1}^T \text{D}_{\text{KL}}(q_{\theta_2}(s_t^{(2)} | s_{<t}^{(2)}, s_{\tau(t):t}^{(1)}) \parallel p_{\theta_2}(s_t^{(2)} | s_{<t}^{(2)})) - \mathcal{L}_{\text{TPC}}(\theta_2) \right], \end{aligned} \quad (12)$$

where $\mathcal{L}_{\text{TPC}}(\theta_2)$ is defined in Eq. (10) and includes both the complete–partial alignment and the margin-based contrastive separation.

4 Related Work

Temporal abstraction in predictive models Temporal abstraction is recognized as a fundamental cognitive ability in animal intelligence [4], and realizing this capability within machine learning has been a long-standing research goal [40, 42, 43]. In time-series modeling, HSSMs have been developed to leverage temporal abstraction to improve long-term predictions. Early work often used fixed timescales, where state transitions occur at predetermined frequencies [22, 27, 30, 35, 39, 41], but performance depends on how well these intervals align with the (often variable) timescales of data. Recent research, therefore, has shifted towards HSSMs with adaptive timescales, which learn or dynamically infer the boundaries of temporal segments from the data itself [6, 8, 11, 25, 26, 46, 49, 50]. For example, VTA [26] introduces latent boundary variables but requires predefined hyperparameters to regulate chunk statistics, while VPR [50] detects boundaries via similarity-based changes and may miss boundaries without salient perceptual shifts. In contrast, our approach detects boundaries independently of these changes. LOVE [25] improves upon VTA by incorporating additional objectives to minimize the code length of high-level latent states. THICK [11] learns adaptive timescales by enforcing L_0 regularization on low-level latent states.

Chunking The cognitive process of organizing repeated information into smaller, meaningful units, known as chunking, is a fundamental mechanism of human learning and memory [5, 32]. The principle of chunking has been widely adopted in compression algorithms [10], sequence modeling [40], representation learning [45], tokenization for large language models [24, 36, 47], and action prediction in robotics [3, 29]. However, despite these advances, no prior work has successfully integrated chunking mechanisms into predictive models for high-dimensional visual inputs.

Predictive coding Predictive coding posits that prediction errors in hierarchical generative models drive learning by continuously comparing predictions with sensory input [38]. Beyond serving

Table 1: **Results on open-loop video prediction.** SSIM and MSE (mean $\pm$ std across 3 random seeds) of each model’s predicted frames against ground-truth frames over 250 timesteps.

	Bouncing Ball		3D Maze		Serial Nine Rooms	
	SSIM $\uparrow$	MSE $\downarrow$	SSIM $\uparrow$	MSE $\downarrow$	SSIM $\uparrow$	MSE $\downarrow$
RSSM	0.911 ± 0.014	1395 ± 25.6	0.635 ± 0.019	7750 ± 25.1	0.686 ± 0.004	2325 ± 72.1
VTA	0.883 ± 0.004	1308 ± 16.2	0.471 ± 0.048	7355 ± 102	0.259 ± 0.029	5648 ± 371
LOVE	0.887 ± 0.010	1285 ± 19.8	0.517 ± 0.027	7194 ± 104	0.206 ± 0.009	5895 ± 373
SUNTA	0.981 ± 0.004	210 ± 27.8	0.933 ± 0.011	580 ± 48.0	0.858 ± 0.025	507 ± 104

as a learning signal, spikes in prediction error have been linked to the boundaries of meaningful events [48]. Early RNN work leveraged prediction failures in a lower-level network to trigger updates in a higher-level network, helping capture long-term dependencies [40]. STM extends this idea to deep generative latent dynamics for visual inputs and improves planning, but it cannot generate all intermediate timesteps during imagination, limiting its use for standard video prediction [49].

5 Experiments

We evaluate SUNTA on two key questions: (1) Is surprise-based chunking effective for discovering temporal boundaries? (2) Does the resulting hierarchical structure improve long-term video prediction over prior methods?

5.1 Datasets

We evaluate SUNTA on three video datasets with natural temporal hierarchies. **Bouncing Ball** is a 2D toy dataset based on Bouncing Balls [26, 50]: a colored ball bounces off walls and changes color at each bounce, with the post-bounce color determined by the sixth previous color, introducing long-range dependencies. **3D Maze** is the egocentric maze navigation dataset from VTA [26], extended with action-conditioned dynamics and wall colors that vary across episodes. **Serial Nine Rooms**, inspired by Pertsch et al. [37] and built on Miniworld [7], places an agent traversing nine serially connected rooms with top-down egocentric observations; each room’s texture is determined by the room visited three rooms earlier (~ 120 timesteps), creating delayed dependencies.

5.2 Baselines

We compare SUNTA against 5 video generative models.

RSSM [13] is a non-hierarchical SSM, used to quantify the benefit of temporal abstraction.

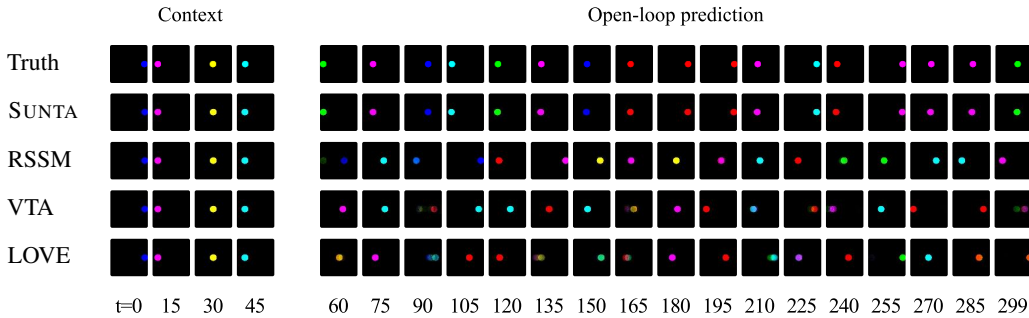


Figure 4: **Qualitative results of open-loop video prediction on the Bouncing Ball dataset.** The top row shows the ground-truth image frames and other rows show the image frames generated by each model up to 250 steps ahead, conditioning on the first 50 context frames.

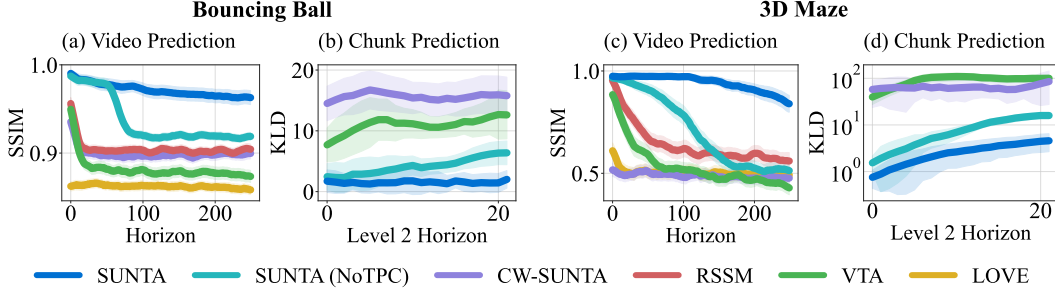


Figure 5: **Open-loop prediction accuracy over the rollout horizon.** We evaluate pixel-level quality via SSIM (higher is better $\uparrow$) and high-level latent-space prediction accuracy via KL divergence (lower is better $\downarrow$). All models are conditioned on the first 50 frames.

VTA [26] is a 2-level HSSM that directly learns the chunk boundary.

VPR [50], which learns a hierarchy via similarity-based chunking, cannot generate frames in an open-loop setting; we therefore use it only for boundary detection.

LOVE [25] augments VTA with an additional compression objective to improve chunk structure.

CW-SUNTA To ablate our chunking mechanism, we introduce a fixed-timescale variant named Clockwork SUNTA, analogous to CW-VAE [39] and CW-VPR [50]. It uses the same architecture and training as SUNTA, but chunk boundaries are placed at predetermined fixed rates. We set the chunk length to 10.

Table 2: **Results on chunk boundary detection.** F1 scores (mean $\pm$ std across 3 random seeds) of the boundaries identified by each model during inference, evaluated against ground-truth boundary points.

	Bouncing Ball	3D Maze
	F1 Score $\uparrow$	F1 Score $\uparrow$
VTA	0.639 ± 0.082	0.577 ± 0.025
VPR	0.467 ± 0.034	0.174 ± 0.039
LOVE	0.276 ± 0.083	0.159 ± 0.008
SUNTA	0.984 ± 0.002	0.992 ± 0.003

5.3 Training and Evaluation Details

All images are resized to 64×64 . Models are trained with the ADOPT optimizer [44] on an NVIDIA H100. We report F1 scores for boundary detection and the Structural Similarity Index (SSIM) and Mean Squared Error (MSE) for video prediction. Quantitative results in Tables 1 and 2 are reported as mean $\pm$ standard deviation over 3 random seeds.

5.4 Results and Discussion

Results on Bouncing Ball We first evaluate the ability of SUNTA to discover temporal structures. For boundary detection, Table 2 and Fig. 3 show that baselines struggle to identify ground-truth boundaries, whereas SUNTA attains near-perfect boundary detection. We next evaluate long-horizon prediction, conditioning on 50 context frames and generating the subsequent 250 frames (Table 1). RSSM fails to capture long-range dependencies, exposing the limitations of flat models. VTA and LOVE, which failed to identify accurate boundaries (Fig. 3), also perform poorly on this task. None of the baselines recover the underlying color-transition rule. In contrast, SUNTA significantly outperforms all baselines. Fig. 4 further illustrates that all baselines degrade within 10 timesteps, while only SUNTA maintains accurate predictions throughout the 250-frame horizon. These results indicate that hierarchical structure significantly affects prediction performance, and that surprise-based chunking is the most reliable mechanism in this setting.

Results on Serial Nine Rooms We further challenge the models on the Serial Nine Rooms dataset, which features more complex textures and longer-range dependencies. Table 1 shows that the gap between SUNTA and the baselines widens further on this dataset: while RSSM degrades substantially and the hierarchical baselines VTA and LOVE collapse altogether, SUNTA maintains a clear margin over all of them. As illustrated in Fig. 6, SUNTA generates coherent room-to-room transitions and remains visually consistent over the full horizon, whereas baselines quickly drift and produce mismatched textures, reflecting their difficulty in maintaining the delayed dependency.

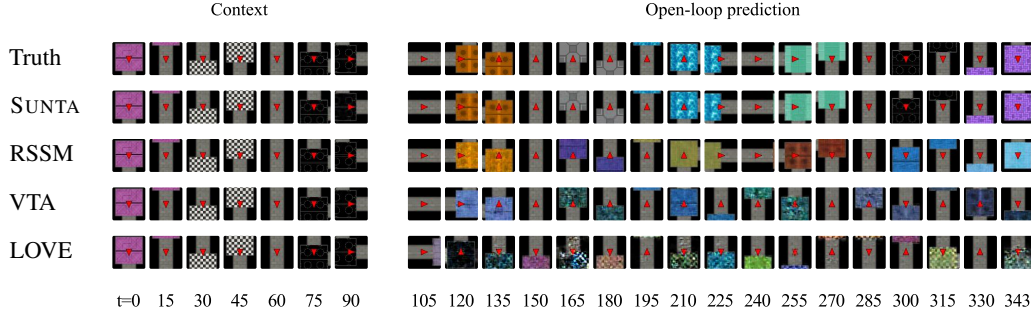


Figure 6: **Qualitative results of open-loop video prediction on the Serial Nine Rooms dataset.** The top row shows the ground-truth image frames and other rows show the image frames generated by each model up to 244 steps ahead, conditioning on the first 100 frames.

Results on 3D Maze Finally, on 3D Maze, SUNTA again dominates both boundary detection (Table 2) and long-horizon prediction (Table 1). Notably, both hierarchical baselines (VTA, LOVE) underperform the non-hierarchical RSSM across all datasets. This suggests that imposing an arbitrary or misaligned hierarchy is not merely ineffective but actively detrimental. SUNTA avoids this failure mode, successfully uncovering meaningful abstractions even under 3D perspective changes; qualitative open-loop predictions on this dataset are shown in Fig. 9 (Section J.2).

Effect of temporal pattern completion. Fig. 5 (SUNTA (NoTPC)) shows that the gap from removing the TPC objective widens with the rollout horizon. We attribute this to the high-level encoder overfitting to complete chunks, yielding unstable representations from partial observations and causing incorrect latent transitions that compound over long horizons.

Fixing chunk length. The fixed-timescale variant CW-SUNTA (i.e., SUNTA without surprise-based chunking) matches the flat RSSM in long-horizon prediction (Table 6, Fig. 5). This indicates that imposing a hierarchy with rigid chunk lengths delivers no gain over a flat SSM, and that the added complexity of a hierarchical mechanism rarely pays for itself unless guided by effective boundaries. The substantial leap achieved by SUNTA, by contrast, demonstrates that the advantage of hierarchical modeling lies not in the architecture itself but in aligning abstraction with the environment’s intrinsic temporal structure, yielding temporal abstractions that simplify the learning of high-level dynamics.

6 Conclusion

We introduced surprise-based nested temporal abstraction (SUNTA), which brings surprise-driven chunking to hierarchical video prediction via a decoupled hierarchy and top-down surprise signals. Experiments show that SUNTA substantially improves long-horizon prediction over prior HSSM baselines, highlighting the importance of aligning hierarchical abstraction with intrinsic temporal structure. These results advance world models for settings that demand accurate long-term foresight.

Limitations SUNTA has several limitations that we hope future work will address. First, the method relies on the presence of meaningful surprise spikes in the low-level model: in environments with uniformly unpredictable dynamics, the surprise signal could lack discriminative peaks and fail to guide effective chunking. Second, our experiments focus on simulated visual environments, and extending SUNTA to real-world, higher-resolution video and multimodal observations is left to future work. Third, while we evaluate up to two levels of hierarchy and find that additional levels are unwarranted on our current benchmarks (Section K), SUNTA’s design naturally extends to deeper hierarchies, and verifying its gains in environments with multi-scale long-range dependencies remains an open question.

Finally, incorporating action generation and decision-making into our temporal abstraction framework represents an exciting avenue for hierarchical planning and control.

References

- [1] Sathyanarayanan N. Aakur and Sudeep Sarkar. A perceptual prediction framework for self supervised event segmentation. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [2] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. In *Advances in Neural Information Processing Systems*, 2024.
- [3] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies, 2025. URL <https://arxiv.org/abs/2506.07339>.
- [4] Matthew M. Botvinick. Hierarchical models of behavior and prefrontal function. *Trends in Cognitive Sciences*, 12(5), 2008.
- [5] William G. Chase and Herbert A. Simon. Perception in chess. *Cognitive Psychology*, 4(1), 1973. ISSN 0010-0285. doi: [https://doi.org/10.1016/0010-0285\(73\)90004-2](https://doi.org/10.1016/0010-0285(73)90004-2).
- [6] Zhengping Che, Sanjay Purushotham, Guangyu Li, Bo Jiang, and Yan Liu. Hierarchical deep generative models for multi-rate multivariate time series. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*. PMLR, 10–15 Jul 2018.
- [7] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *CoRR*, abs/2306.13831, 2023.
- [8] Junyoung Chung, Sungjin Ahn, and Yoshua Bengio. Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations*, 2017.
- [9] Karl Friston, Rosalyn J. Moran, Yukie Nagai, Tadahiro Taniguchi, Hiroaki Gomi, and Josh Tenenbaum. World model learning and inference. *Neural Networks*, 144, 2021. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2021.09.011>.
- [10] Philip Gage. A new algorithm for data compression. *C Users J.*, 12(2), February 1994. ISSN 0898-9788.
- [11] Christian Gumbsch, Noor Sajid, Georg Martius, and Martin V. Butz. Learning hierarchical world models with adaptive temporal abstractions from discrete latent dynamics. In *International Conference on Learning Representations*, 2024.
- [12] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [13] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, 2019.
- [14] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- [15] Danijar Hafner, Timothy P Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.
- [16] Danijar Hafner, Kuang-Huei Lee, Ian Fischer, and Pieter Abbeel. Deep hierarchical planning from pixels. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [17] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

- [18] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025. URL <https://arxiv.org/abs/2509.24527>.
- [19] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.
- [20] Nicklas A Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, 2022.
- [21] Jeff Hawkins and Sandra Blakeslee. *On Intelligence*. Times Books, 2004. ISBN 0805074562.
- [22] Salah Hihi and Yoshua Bengio. Hierarchical recurrent neural networks for long-term dependencies. In *Advances in Neural Information Processing Systems*, volume 8, 1995.
- [23] Jakob Hohwy. *The Predictive Mind*. Oxford University Press, 2013.
- [24] Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling, 2025. URL <https://arxiv.org/abs/2507.07955>.
- [25] Yiding Jiang, Evan Liu, Benjamin Eysenbach, J. Zico Kolter, and Chelsea Finn. Learning options via compression. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [26] Taesup Kim, Sungjin Ahn, and Yoshua Bengio. Variational temporal abstraction. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [27] Jan Koutnik, Klaus Greff, Faustino Gomez, and Juergen Schmidhuber. A clockwork rnn. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32, 2014.
- [28] Yann LeCun. A path towards autonomous machine intelligence. *OpenReview Archive Preprint*, 2022.
- [29] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking, 2025. URL <https://arxiv.org/abs/2507.07969>.
- [30] Paul Mattes, Rainer Schlosser, and Ralf Herbrich. Hieros: Hierarchical imagination on structured state space sequence world models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, 2024.
- [31] Johan Medrano, Karl Friston, and Peter Zeidman. Linking fast and slow: The case for generative models. *Network Neuroscience*, 8, 2024.
- [32] George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *The Psychological Review*, 63(2), March 1956.
- [33] Alireza Modirshanechi, Johanni Brea, and Wulfram Gerstner. A taxonomy of surprise definitions. *Journal of Mathematical Psychology*, 110, 2022. ISSN 0022-2496. doi: <https://doi.org/10.1016/j.jmp.2022.102712>.
- [34] Ramy Mounir, Sujal Vijayaraghavan, and Sudeep Sarkar. Streamer: Streaming representation learning and event segmentation in a hierarchical manner. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023.
- [35] Asier Mujika, Florian Meier, and Angelika Steger. Fast-slow recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [36] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens, 2024. URL <https://arxiv.org/abs/2412.09871>.

- [37] Karl Pertsch, Oleh Rybkin, Frederik Ebert, Chelsea Finn, Dinesh Jayaraman, and Sergey Levine. Long-horizon visual planning with goal-conditioned hierarchical predictors. *arXiv preprint arXiv:2006.13205*, 2020.
- [38] Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 1999. doi: 10.1038/4580.
- [39] Vaibhav Saxena, Jimmy Ba, and Danijar Hafner. Clockwork variational autoencoders. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [40] Jürgen Schmidhuber. Learning complex, extended sequences using the principle of history compression. *Neural Computation*, 4(2), 1992. doi: 10.1162/neco.1992.4.2.234.
- [41] Vaisakh Shaj, Saleh Gholam Zadeh, Ozan Demir, Luiz Ricardo Douat, and Gerhard Neumann. Multi time scale world models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [42] Richard S. Sutton. TD Models: Modeling the World at a Mixture of Time Scales. In *Proceedings of the 12th International Conference on Machine Learning*. Morgan Kaufmann, 1995. ISBN 1-55860-377-8.
- [43] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1), 1999.
- [44] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Naga-hara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. Adopt: Modified adam can converge with any β_2 with the optimal rate. In *Advances in Neural Information Processing Systems*, 2024.
- [45] Shuchen Wu, Noemi Elteto, Ishita Dasgupta, and Eric Schulz. Learning structure from the ground up—hierarchical representation learning by chunking. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [46] Yuichi Yamashita and Jun Tani. Emergence of functional hierarchy in a multiple timescale neural network model: A humanoid robot experiment. *PLOS Computational Biology*, 4, 2008. doi: 10.1371/journal.pcbi.1000220.
- [47] LILI YU, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. Megabyte: Predicting million-byte sequences with multiscale transformers. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023.
- [48] Jeffrey M Zacks, Nicole K Speer, Khen M Swallow, Todd S Braver, and Jeremy R Reynolds. Event perception: a mind-brain perspective. *Psychological bulletin*, 133(2), March 2007. ISSN 0033-2909. doi: 10.1037/0033-2909.133.2.273.
- [49] Alexey Zakharov, Matthew Crosby, and Zafeirios Fountas. Episodic memory for subjective-timescale models. In *ICML 2021 Workshop on Unsupervised Reinforcement Learning*, 2021.
- [50] Alexey Zakharov, Qinghai Guo, and Zafeirios Fountas. Variational predictive routing with nested subjective timescales. In *International Conference on Learning Representations*, 2022.

A Hyperparameters

Table 3 summarizes the hyperparameters used in our main experiments. Unless otherwise stated, the same values are used across all datasets.

Table 3: **Hyperparameters used in the experiments.**

Hyperparameter	Value
Optimizer	ADOPT [44]
Learning rate	3×10^{-4}
Batch size	64
Training steps (per level)	250,000
Gradient clipping	10.0
Image resolution	64×64
Recurrent (deterministic) state dim	512
Stochastic state dim ($s_t^{(1)}, s_t^{(2)}$)	32
Peak-detection prominence	0.1
TPC margin γ	50
Loss weight: reconstruction (NLL)	1.0
Loss weight: state KL (level-1 posterior–prior)	1.0
Loss weight: state reconstruction KL (level-2 to level-1)	1.0
Loss weight: TPC alignment (complete–partial KL)	1.0
Loss weight: TPC contrastive separation (margin term)	1.0

B Architecture Details

Vision encoder and decoder. We adopt a DreamerV2-style convolutional encoder–decoder for all models. The encoder takes a $3 \times 64 \times 64$ RGB frame and applies four Conv2d blocks with kernel size 4 and strides (2, 2, 2, 1), each followed by BatchNorm2d and ELU activation, producing a C -dimensional embedding (channel width $C = 64$). The decoder mirrors this with four ConvTranspose2d blocks with kernel size 4 and strides (1, 2, 2, 2) and BatchNorm2d+ELU.

RSSM internals. Each level is implemented as an RSSM [13] with the following components:

- Recurrent backbone: GRUCell.
- Latent distribution: diagonal Gaussian; the standard deviation is parameterized through a Sigmoid activation.
- Activation: ELU throughout the MLP heads.

The level-1 decoder takes the stochastic latent $s_t^{(1)}$ as input. The level-2 top-down decoder takes the concatenation of the deterministic and stochastic components of $s_t^{(2)}$.

C Pseudocode

SUNTA is trained in two stages following the decoupled level-wise scheme described in Section 3. Algorithm 1 summarizes the procedure.

D Boundary Detection

Surprise computation. We compute the inference-time surprise as the reverse KL divergence between the level-1 posterior and prior at each timestep (Section 3). We use the reverse formulation

Algorithm 1 SUNTA

```
1: Stage 1: train the low-level model.
2: Initialize  $\theta_1$ .
3: for step = 1, ..., 250,000 do
4:   Sample a batch of observation sequences  $(x_{1:T})$ .
5:   Compute the level-1 ELBO and update  $\theta_1$ .
6: end for
7: Freeze  $\theta_1$ .

8: Stage 2: train the high-level model.
9: Initialize  $\theta_2$ .
10: for step = 1, ..., 250,000 do
11:   Sample a batch and infer  $s_{1:T}^{(1)}$  via the frozen  $q_{\theta_1}$ .
12:   Compute the low-level surprise sequence and detect boundaries  $m_{1:T}$  via find_peaks.
13:   Sample a partial-chunk prefix and a negative-chunk context for the TPC regularizer.
14:   Compute the level-2 ELBO with  $\mathcal{L}_{\text{TPC}}$  and update  $\theta_2$ .
15: end for

16: Open-loop generation.
17: Given a context segment, infer  $s_t^{(2)}$  for the most recent chunk.
18: for rollout step  $t' > t$  do
19:   Decode top-down to obtain  $\hat{s}_{t'}^{(1)}$  and roll the low-level prior forward.
20:   Compute the top-down surprise and detect chunk boundaries  $\tilde{m}_{t'}$  via find_peaks.
21:   if a boundary is detected then
22:     Transition the high-level state via the level-2 prior.
23:   end if
24: end for
```

$D_{\text{KL}}(q \parallel p)$ rather than $D_{\text{KL}}(p \parallel q)$ because it more reliably detects shifts in the posterior toward unexpected outcomes.

Peak detection. We use SciPy’s `find_peaks` on the per-sequence surprise signal with the prominence parameter. Surprise signals are first normalized by their running maximum to remove scale variation across sequences. At the end of each sequence we append a sentinel value (the running mean of the signal) so that genuine surprise spikes at the final timestep are not missed by `find_peaks`.

Hyperparameter sensitivity and choice of prominence. SUNTA introduces a single peak-detection hyperparameter, the prominence parameter, used both during training to identify boundaries from low-level surprise (the threshold τ_{inf}) and during open-loop generation to detect chunk switches from top-down surprise (the threshold τ_{gen}). Table 4 reports the F1 of detected boundaries and the SSIM of open-loop video prediction across a wide range of prominence values on Bouncing Ball and 3D Maze.

We choose `prominence` = 0.1 for all main experiments because it sits near the center of the stable range [0.05, 0.3] and transfers without retuning to Serial Nine Rooms, where ground-truth boundaries are unavailable for direct tuning. Outside this range, we observe the failure modes consistent with their respective limits: very small values (e.g., 0.001) trigger spurious peaks and over-segment the sequence, while very large values (e.g., 1.0) miss genuine peaks and under-segment. Both behaviors degrade downstream prediction quality.

E Baseline Implementation

All baselines use the same vision encoder, decoder, recurrent backbone, and hidden dimensions as SUNTA unless otherwise stated.

Table 4: **Sensitivity to peak-detection prominence.** F1 score of detected boundaries during training (left) and SSIM of open-loop video prediction (right) on Bouncing Ball and 3D Maze, varying the prominence parameter of SciPy’s `find_peaks`. Performance is stable across a wide range; we use 0.1 in all main experiments.

prominence	F1 score $\uparrow$		SSIM $\uparrow$	
	Bouncing Ball	3D Maze	Bouncing Ball	3D Maze
0.001	0.712	0.834	0.917	0.706
0.01	0.989	0.989	0.955	0.818
0.05	0.994	0.996	0.986	0.940
0.1	0.995	0.996	0.986	0.942
0.3	0.995	0.996	0.984	0.927
0.5	0.978	0.982	0.935	0.693
1.0	0.706	0.853	0.903	0.491

RSSM [13]. A non-hierarchical baseline using only the level-1 components of SUNTA. To provide a stronger non-hierarchical baseline, we increase RSSM’s capacity by setting the deterministic-state dimension to 2048 and the stochastic-state dimension to 64.

VTA [26]. A two-level HSSM with learned binary boundary variables. We use the authors’ official implementation, with deterministic-state size 512 and stochastic-state size 32 at both levels. We set the maximum number of subsequences $N_{\max}$ to 20 and maximum subsequence length $h_{\max}$ to 15 .

LOVE [25]. Built on top of VTA with an additional compression objective. We use the authors’ official implementation, with a discrete latent codebook of size 30 and a coding-length coefficient of 0.1.

VPR [50]. We re-implemented VPR following the original paper, using similarity-based chunking. Because VPR cannot generate frames in an open-loop setting, we report it only for boundary detection.

CW-SUNTA. A fixed-timescale ablation of SUNTA. The architecture and training procedure are identical to SUNTA, but chunk boundaries are placed at fixed intervals with chunk length 10.

F Compute Resources

All experiments were run on a single NVIDIA H100 GPU. Training each level for 250,000 steps takes approximately 48 hours; full SUNTA training (2-level) therefore takes approximately 96 hours per seed.

G Parameter Count

We compare SUNTA’s parameter count against the baselines (Table 5). SUNTA achieves its gains with fewer trainable parameters than all baselines, indicating that its improvements arise from the surprise-driven chunking mechanism rather than from additional model capacity.

This efficiency stems from SUNTA’s decoupled, inter-level inference (Section 3). Existing HSSMs infer latent states at all levels directly from the raw observations, so every level must process high-dimensional inputs. In contrast, SUNTA’s high-level model treats the already-inferred low-level latents as its observations and never processes the raw inputs, allowing it to achieve accurate long-horizon prediction with substantially less model capacity.

Table 5: **Parameter count comparison.** Trainable parameters at a fixed hidden dimension.

Model	# Parameters ↓
RSSM (flat)	30.74M
VTA (2-level)	44.26M
LOVE (2-level)	44.21M
SUNTA (2-level)	15.40M

H Temporal Pattern Completion

Fig. 7 illustrates the train–test mismatch addressed by temporal pattern completion (TPC) and our regularization scheme (Section 3). At training time, the high-level encoder q_{θ_2} aggregates the entire low-level latent sequence within each chunk (Fig. 7a, top) to produce the chunk representation $s_t^{(2)}$. At test time, however, observations are revealed online: the most recent chunk in the context is typically truncated, and during open-loop generation the high-level state must be inferred from only a partial low-level rollout (Fig. 7a, bottom). Without explicit regularization, this discrepancy yields representations that drift away from those seen at training and propagate errors over the open-loop horizon.

To simulate this condition during training, we draw a prefix length uniformly within each chunk and feed the prefix through the high-level encoder to obtain the partial posterior $q_{\theta_2}^p$ in addition to the complete posterior $q_{\theta_2}^c$ (Fig. 7b). The KL penalty in Eq. (10) aligns the partial posterior with the complete one, while the contrastive margin term against a negative posterior $q_{\theta_2}^n$ drawn from an unrelated chunk prevents collapse to a chunk-invariant representation.

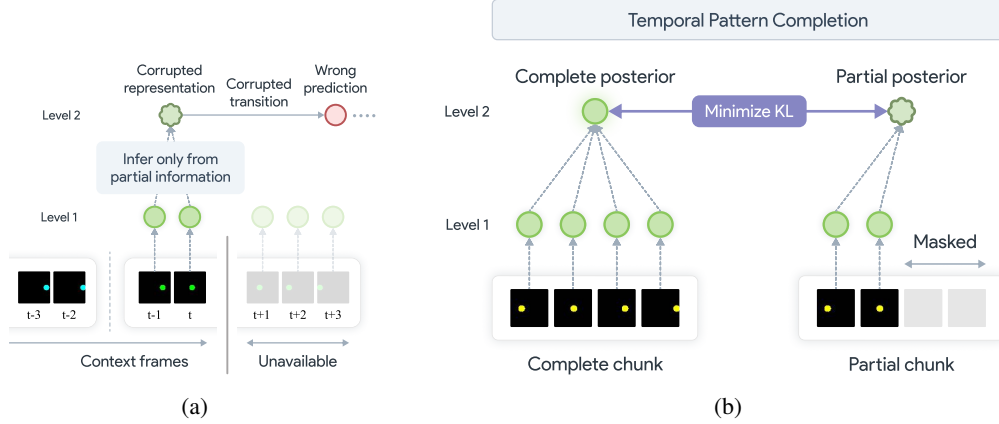


Figure 7: (a) **Incomplete chunks at the end of context.** During online inference, the context (observations available up to the current timestep) often ends with a truncated chunk. Since the high-level model is trained on complete chunks, this train–test mismatch can lead to flawed high-level representations and degrade subsequent prediction during the rollout. (b) **Temporal pattern completion (TPC) regularization.** To mitigate the train–test mismatch caused by partial chunks, we introduce TPC regularization for the high-level model. During training, we simulate a partial chunk by masking the final timesteps of a chunk. We then enforce representational consistency by minimizing the KL divergence between the high-level state inferred from the partial chunk and the state inferred from the complete chunk.

I Ablation Details

We conduct two complementary ablations on the Bouncing Ball dataset, isolating SUNTA’s three core components: surprise-based chunking, decoupled level-wise training, and temporal pattern completion (TPC).

Independent ablation. Table 6 reports prediction SSIM when each component is removed individually from the full SUNTA model. Removing surprise-based chunking (yielding the fixed-timescale variant CW-SUNTA) or TPC each causes a notable drop, while removing decoupled level-wise training is the most damaging single change.

Table 6: **Independent ablation on Bouncing Ball.** Open-loop video prediction SSIM when each core component is removed individually from the full SUNTA model.

	Bouncing Ball SSIM $\uparrow$
SUNTA (full)	0.986
w/o surprise-based chunking (i.e., CW-SUNTA)	0.918
w/o temporal pattern completion (i.e., NoTPC)	0.936
w/o decoupled level-wise training	0.880

Incremental ablation. Table 7 starts from a non-hierarchical RSSM and successively adds each component. Naively introducing a two-level hierarchy—whether with fixed chunks or with surprise-based chunking—under end-to-end training degrades performance below the flat RSSM, consistent with our hypothesis of hierarchical collapse. Switching to decoupled level-wise training reverses this trend, and adding TPC yields a further substantial gain. This pattern shows that surprise-based chunking and a two-level hierarchy on their own are insufficient: preventing hierarchical collapse via decoupled training is essential, and TPC further closes the partial-chunk train–test gap.

Table 7: **Incremental ablation on Bouncing Ball.** Open-loop video prediction SSIM as components are added on top of a non-hierarchical RSSM baseline.

	Bouncing Ball SSIM $\uparrow$
RSSM (flat)	0.904
+ End-to-end two-level RSSM (fixed chunk length)	0.889
+ Surprise-based chunking	0.883
+ Decoupled level-wise training	0.936
+ Temporal pattern completion (= SUNTA)	0.986

Effect of decoupled training on chunk discovery. To diagnose hierarchical collapse, we visualize the chunks discovered by SUNTA when trained end-to-end versus with the proposed decoupled procedure (Fig. 8). End-to-end training collapses to a degenerate segmentation that ignores the underlying color transitions, whereas decoupled training preserves the surprise signal and recovers the ground-truth boundaries. This is consistent with the corresponding quantitative gain in Table 7.

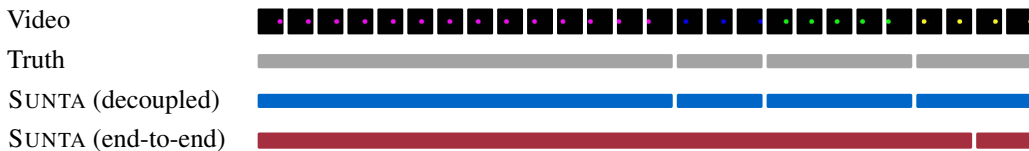


Figure 8: **Effect of decoupled training on chunk discovery (Bouncing Ball).** End-to-end training collapses into a degenerate segmentation, whereas decoupled level-wise training preserves the surprise signal and recovers the ground-truth boundaries (where the ball bounces and changes color).

J Additional Results

J.1 Generality across Backbones

While we use GRU-based variational inference to match existing HSSM baselines, the fundamental concept of using prediction errors to form temporal chunks and learning chunk-level transitions with a separate model could be implemented with alternative backbone architectures. To probe this, we replace the GRU in both levels with xLSTM [2], a more recent recurrent architecture. Table 8 shows that the gains of SUNTA are preserved and slightly improved under this replacement, suggesting that surprise-based chunking transfers across recurrent backbones. Extending SUNTA to non-recurrent backbones such as Transformers is an interesting direction for future work.

Table 8: **Generality across recurrent backbones.** Open-loop video prediction on Bouncing Ball when replacing the GRU in both levels with xLSTM [2].

	SSIM $\uparrow$	MSE $\downarrow$
SUNTA w/ GRU	0.986	185
SUNTA w/ xLSTM	0.992	103

J.2 Video Prediction on 3D Maze

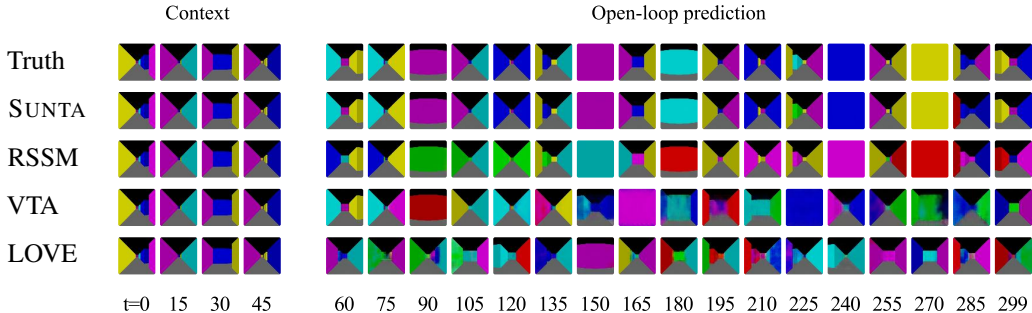


Figure 9: **Qualitative results of open-loop video prediction on the 3D Maze dataset.** The top row shows the ground-truth image frames and other rows show the image frames generated by each model up to 250 steps ahead, conditioning on the first 50 context frames.

J.3 Prediction accuracy within high-level latent space

To verify that our chunking mechanism simplifies the learning of high-level dynamics, we evaluate prediction accuracy solely within the high-level latent space. We measure the KL divergence between the ground-truth posteriors and the predicted priors obtained via recursive “jumpy” rollouts, thereby isolating the accuracy of chunk predictions from the image decoding quality or boundary detection errors during generation. As illustrated in Fig. 5, baseline models exhibit consistently large KL divergence. Recalling their failure to identify correct boundaries (Table 2), these results suggest that inappropriate chunking forces the high level to model incoherent transitions, making the dynamics difficult to capture even when the underlying temporal patterns are simple. Conversely, SUNTA maintains low KL divergence throughout the rollout. This confirms that surprise-based chunking successfully simplifies the explanation of chunk transitions, yielding a tractable abstraction that directly translates to superior long-term prediction.

K Determining the Number of Hierarchical Levels

A natural question is whether deeper hierarchies could yield further gains. In SUNTA, an additional level is warranted only if the current top level itself exhibits unpredicted events, since each higher

level is intended to model longer-range transitions over the chunks discovered by the level below. Fig. 10 visualizes the level-2 surprise signal across Bouncing Ball trajectories: once the high-level model has observed sufficient color history (six bounces), its surprise stays consistently low and nearly flat, indicating that a third level would have little to model in this setting. We observe the same trend on the other datasets and therefore restrict SUNTA to two levels in this work, leaving the study of deeper hierarchies in environments with multi-scale long-range dependencies to future work.

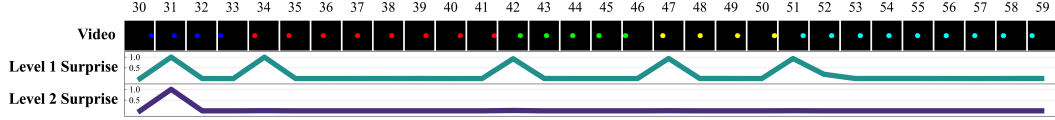


Figure 10: **Surprise signals on Bouncing Ball.** Uniquely determining future ball colors requires observing the previous six bounces. Once sufficient color history has been observed, the level-2 model is no longer surprised and predicts the future correctly, yielding near-zero surprise. The level-1 model continues to be surprised at each bounce, producing the spikes that define chunk boundaries.